VISUAL NOTES • COMPLETE GUIDE

Visual Notes — Specification-Driven AI Coding

Specification-Driven AI Coding

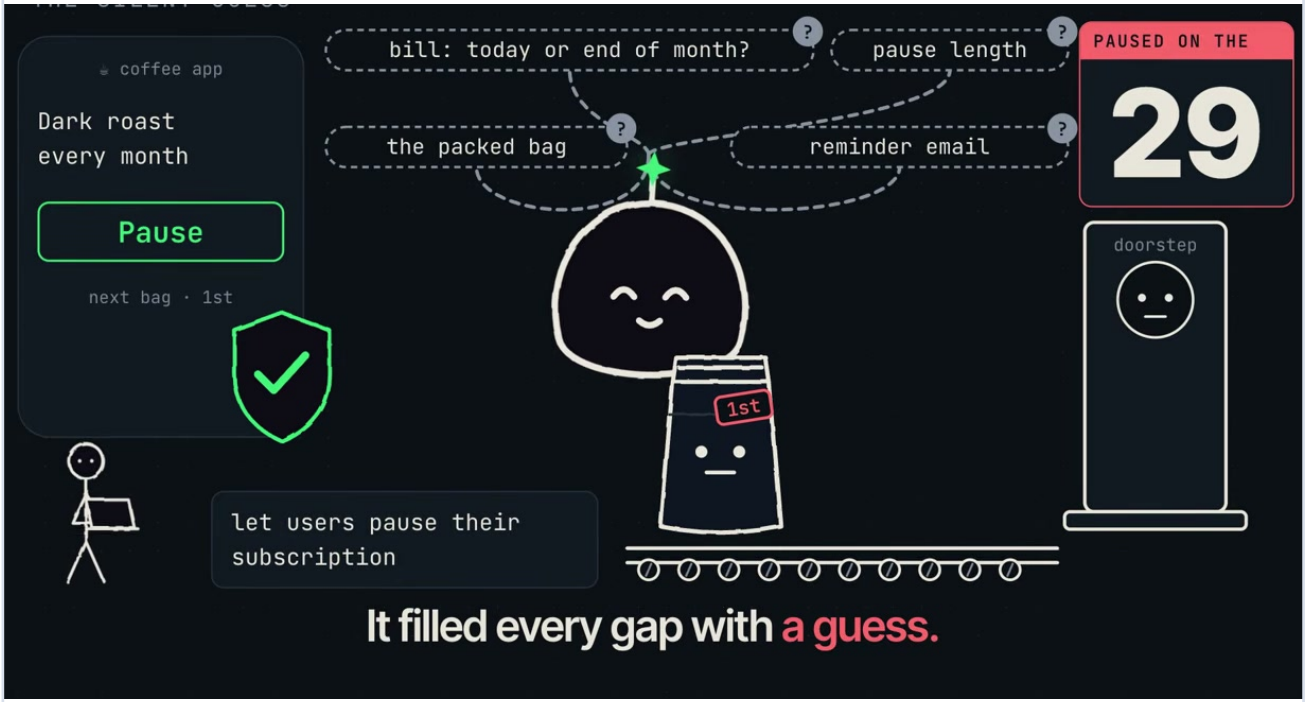
09:05

- 28 substantive visual(s), selected by content
- Repeated builds, presenter-only shots, and filler excluded
- Output length expands or contracts with the source material
- Claims, uncertainty, and caveats are labeled clearly

https://www.youtube.com/watch?v=Q3uuE2adUT8

The Silent Guess

A working feature can still encode unstated product decisions



NOTES

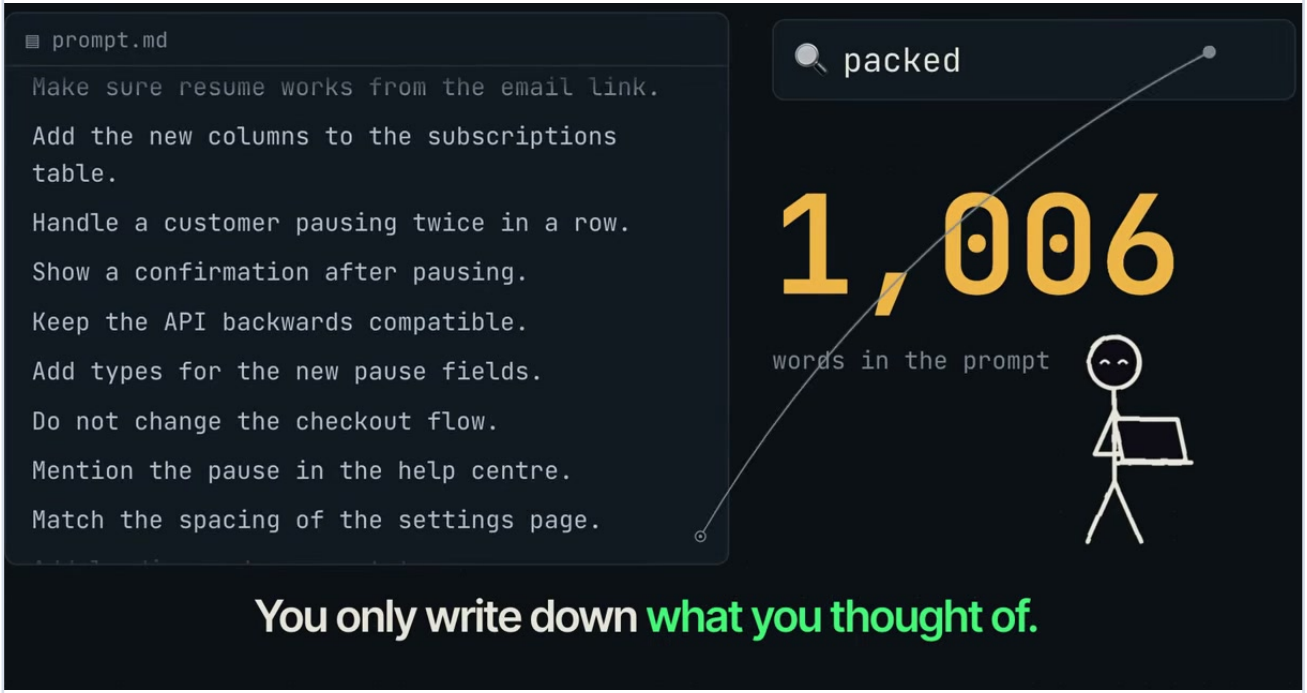
- The coffee pause is an illustration: a one-sentence request produces a Pause button and passing tests, but it does not specify what happens to a bag already roasted and packed.
- The agent silently chooses billing timing, maximum pause length, packed-bag handling, and reminder wording so it can keep moving.
- Those choices are product policy, not implementation trivia; a technically correct build can therefore be operationally wrong.
- Automation changes the human role from typing code to making the decisions the code must preserve.

CORE IDEA

Unstated policy does not disappear; it becomes an agent guess.

Longer Prompts Still Miss Unknowns

More instructions only capture decisions already in mind



NOTES

- A thousand-word prompt can enumerate tests, styling, analytics, database changes, and compatibility while still omitting the packed-bag edge case.
- Prompt length is not decision completeness: people write down what they remembered, not every branch hidden in the feature.
- The corrective move is procedural—make the agent ask before coding—rather than trying to author a perfect brief unaided.
- Treat this example as a product-design illustration, not evidence that a particular prompt length is universally insufficient.

CORE IDEA

Coverage comes from systematic discovery, not raw prompt size.

Start With a Discovery Interview

Invoke /grill-me before implementation



NOTES

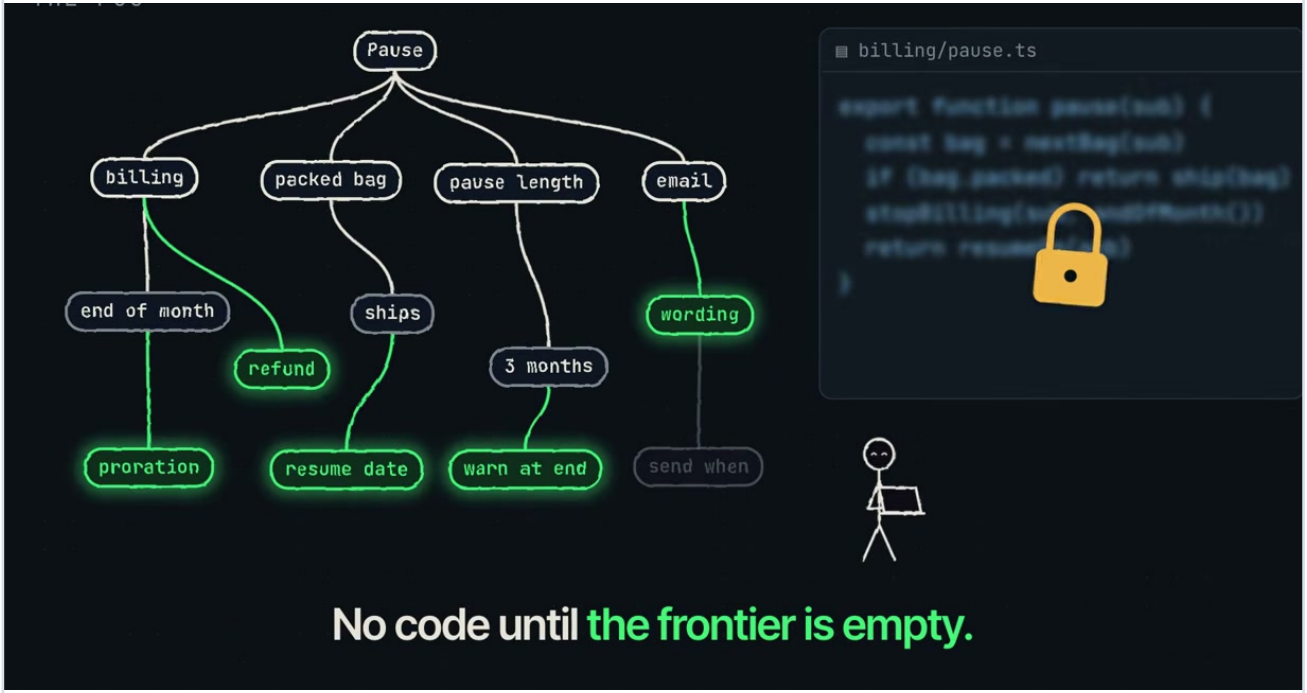
- The concrete command is `/grill-me`: a structured interview intended to sharpen a plan or design before any code exists.
- The interaction is inverted: instead of the human anticipating every edge case, the agent probes the idea for missing choices.
- Questions should separate repository facts from genuine decisions; facts can be researched, while policy calls stay with the human.
- Do not authorize implementation until the parties explicitly agree that the decision set is complete.

CORE IDEA

Interviewing exposes requirements that prompt drafting leaves invisible.

Clear the Decision Frontier

Ask every currently answerable branch, then reveal the next layer



NOTES

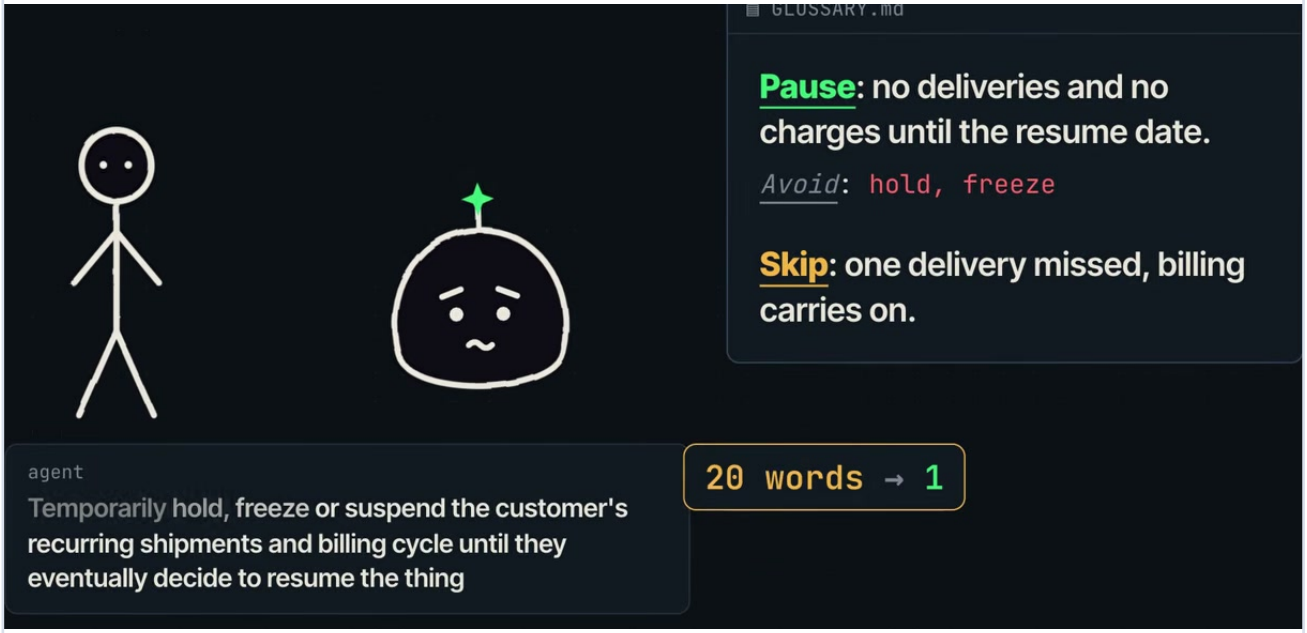
- The decision frontier is the set of unresolved choices whose parent decisions are already settled; it is the next safe batch of questions.
- For a pause feature, branches include billing, packed inventory, duration, resume timing, refunds, email timing, and wording.
- Ask the whole frontier as one numbered round, preferably with recommended answers so the human can reply compactly—for example, `yes, yes, no, end of month`.
- After each round, answers reveal a new frontier; code remains locked until no unresolved branch remains and shared understanding is confirmed.
- Reported question counts vary with scope; figures such as roughly six for a small change, twenty for a normal one, or a 46-question outlier are examples, not targets.

CORE IDEA

A frontier turns fuzzy discovery into a finite, inspectable process.

Give Each Concept One Name

A repository glossary removes semantic drift



NOTES

- `GLOSSARY.md` defines Pause as no deliveries and no charges until the resume date; Skip means one missed delivery while billing continues.
- It also records discouraged synonyms such as hold and freeze, preventing the same concept from acquiring competing names.
- A glossary lets agents and humans use one compact term instead of repeatedly inventing long, ambiguous descriptions.
- Update definitions during discovery so later specs, tickets, tests, and code all inherit the same language.

CORE IDEA

Stable vocabulary reduces both token waste and policy ambiguity.

Rewrite Confusion in Project Language

Use /wait-what for a plain-language restatement



NOTES

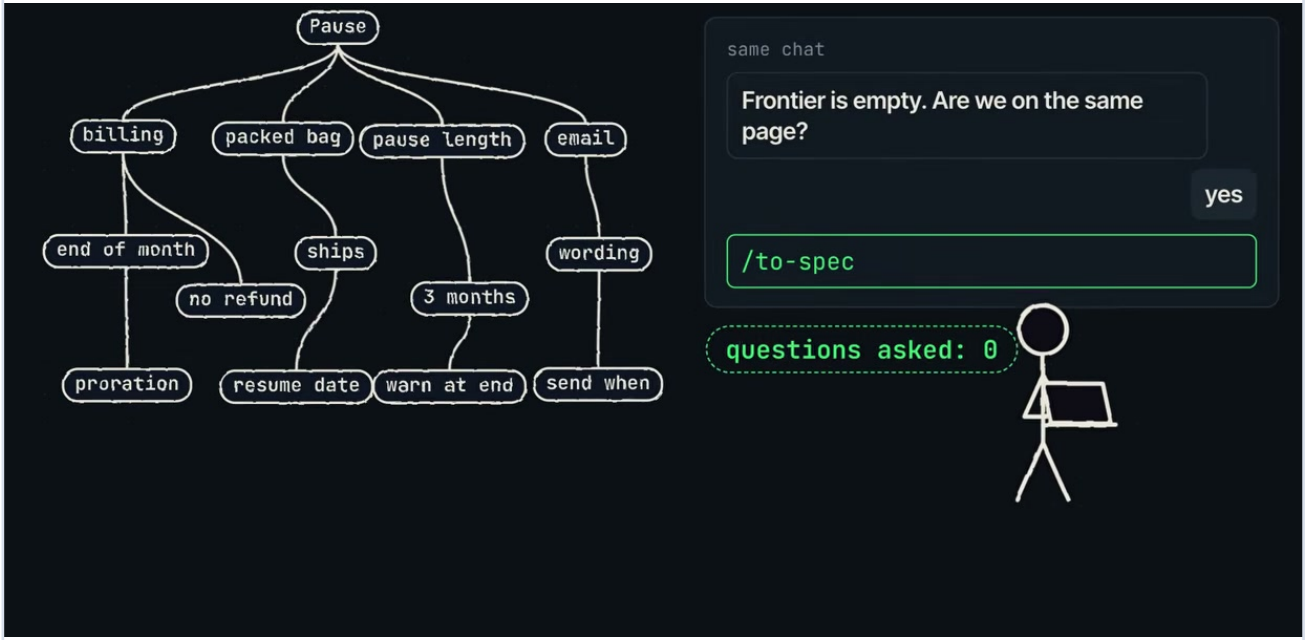
- When an answer becomes abstract or jargon-heavy, type `/wait-what` rather than pretending it is clear.
- The skill restates the previous message in plain English and uses definitions already captured in the glossary.
- The illustrated rewrite—`Pause is the rule everything else depends on`—replaces a vague phrase with a concrete dependency statement.
- This is a comprehension checkpoint, not a new source of product decisions; unresolved choices still return to discovery.

CORE IDEA

Shared language is useful only when every participant can understand it.

Prototype Ungrillable Decisions

Test visual choices, then return the answer to the same decision record



NOTES

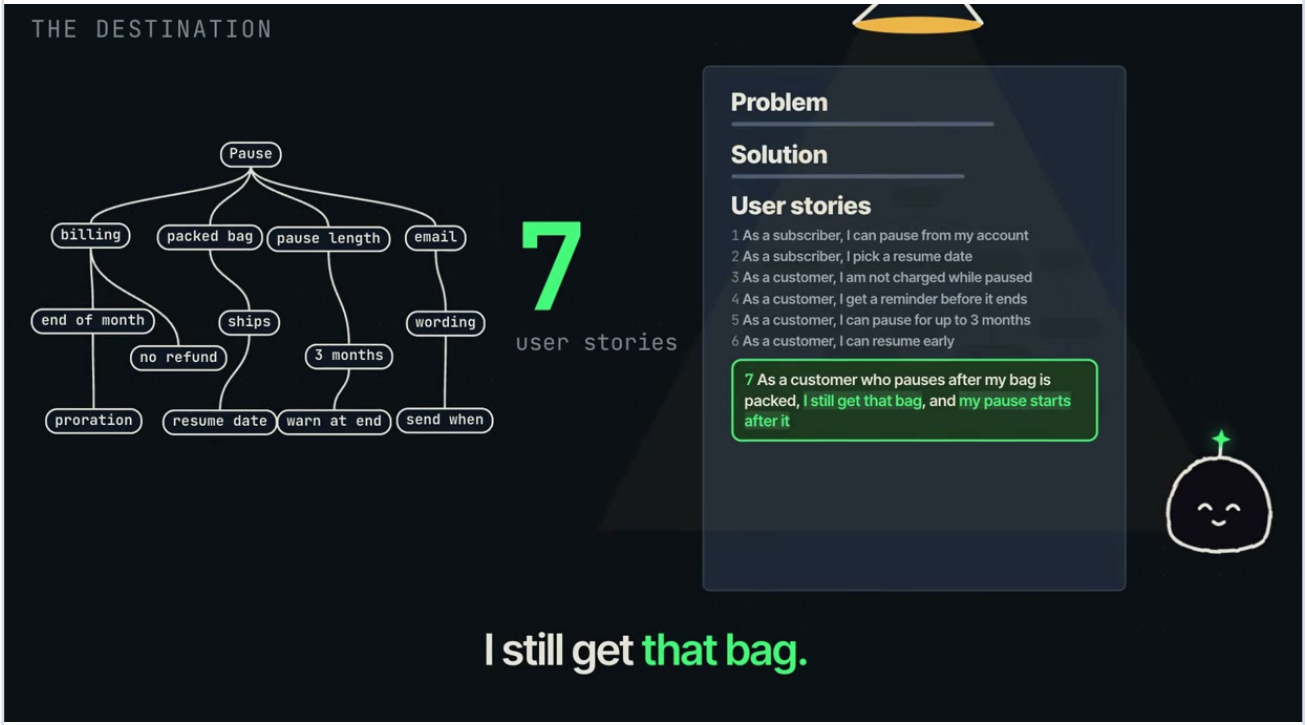
- Some questions—such as what the pause screen should look like—cannot be resolved reliably through verbal interrogation alone.
- Hand those questions to a fresh context: an isolated session with no accumulated conversation, used here to build several deliberately different throwaway screens.
- Click through the prototypes, choose an option, and feed the decision back into discovery; the prototype is evidence, not production code.
- When the frontier is empty and alignment is explicit, stay in the decision chat and run `/to-spec`; this step should organize settled answers rather than reopen discovery.

CORE IDEA

Cheap prototypes convert subjective debate into an observable choice.

Turn Decisions Into User Stories

The spec records the problem, solution, and observable outcomes



NOTES

- A spec is the temporary, structured destination for agreed decisions: the problem, proposed solution, and numbered user stories that define observable behavior.
- The packed-bag story is explicit: if a customer pauses after packing, that bag still ships and the pause starts afterward.
- Numbered stories make omissions reviewable and give later tickets and tests stable references.
- The spec describes outcomes rather than a long implementation recipe; technical decomposition comes next.

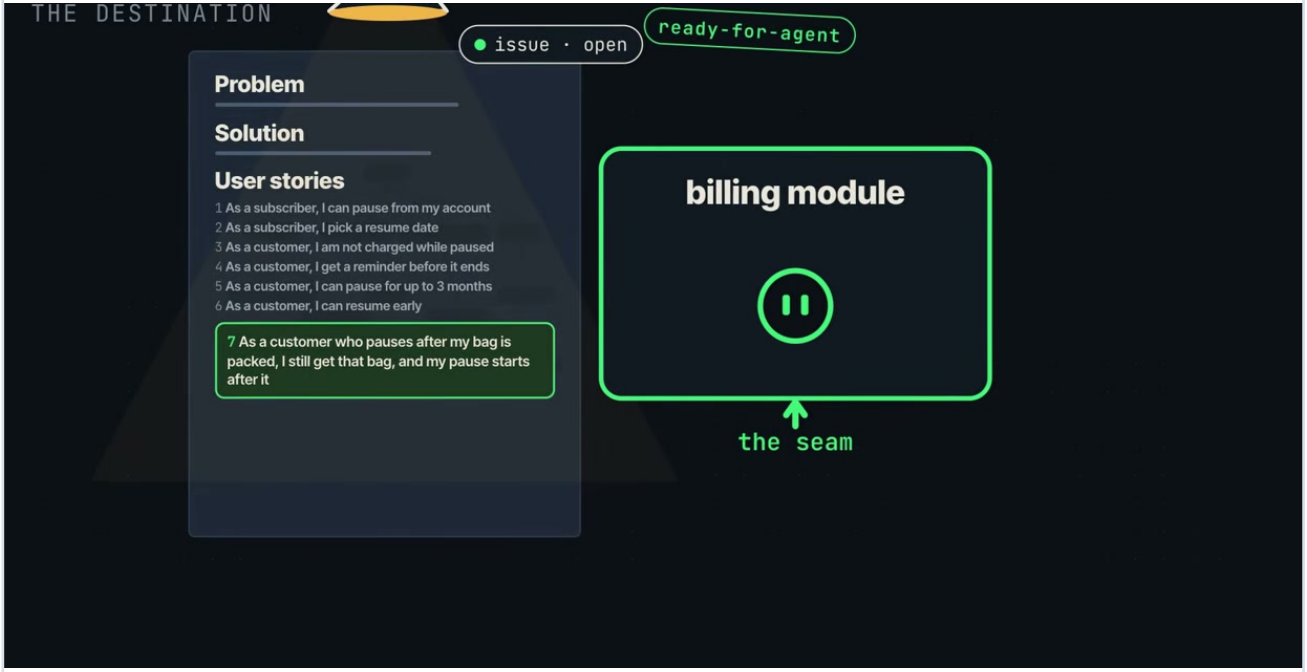
CORE IDEA

A spec makes the decision record testable without turning it into code.

Source: https://www.youtube.com/watch?v=Q3uuE2adUT8

Agree on the Test Seam

Publish the spec only after choosing where behavior will be observed



NOTES

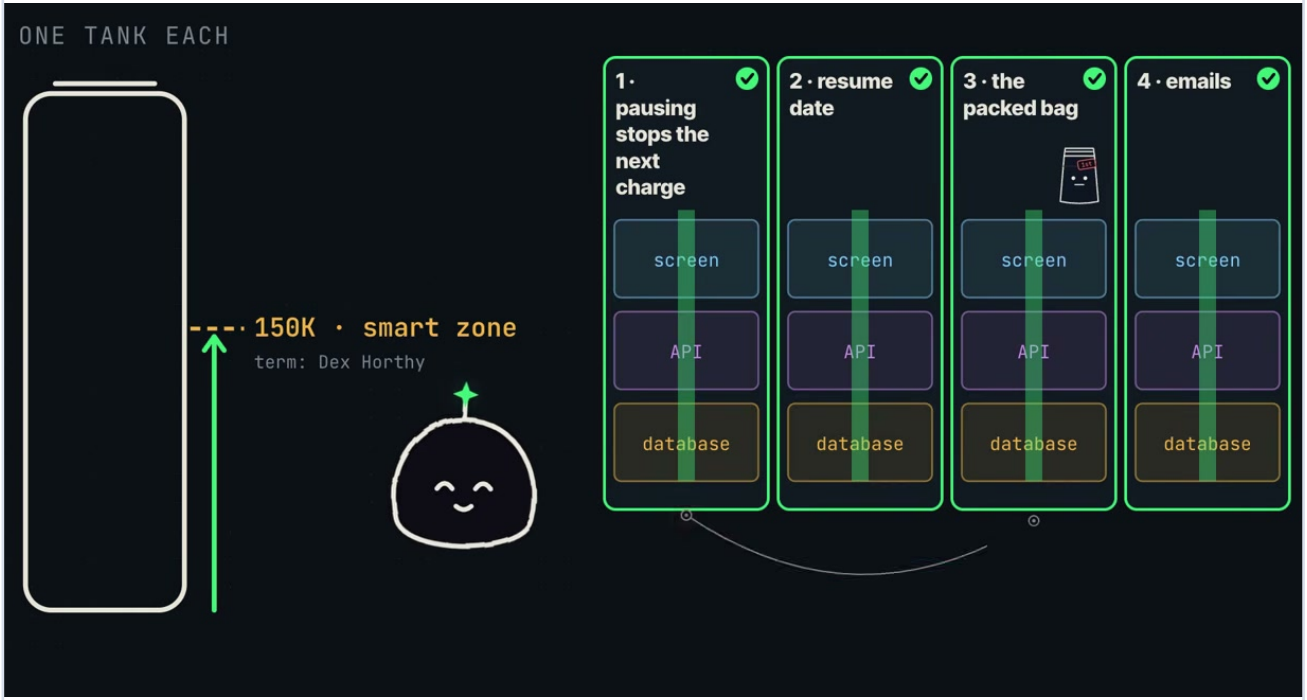
- A seam is a stable boundary where tests can exercise or observe behavior without depending on every internal detail.
- For the illustration, the billing module is the seam for pause behavior; the stated ideal is one primary seam when that is practical.
- Once the seam and stories are agreed, publish the spec as a tracker issue and label it `ready-for-agent`.
- One seam is a heuristic, not a law: complex systems may require several boundaries, and forcing one can hide real integration risk.

CORE IDEA

An agreed seam connects product intent to executable proof.

Slice the Journey Into Vertical Tickets

Each ticket gets a bounded context and crosses every required layer



NOTES

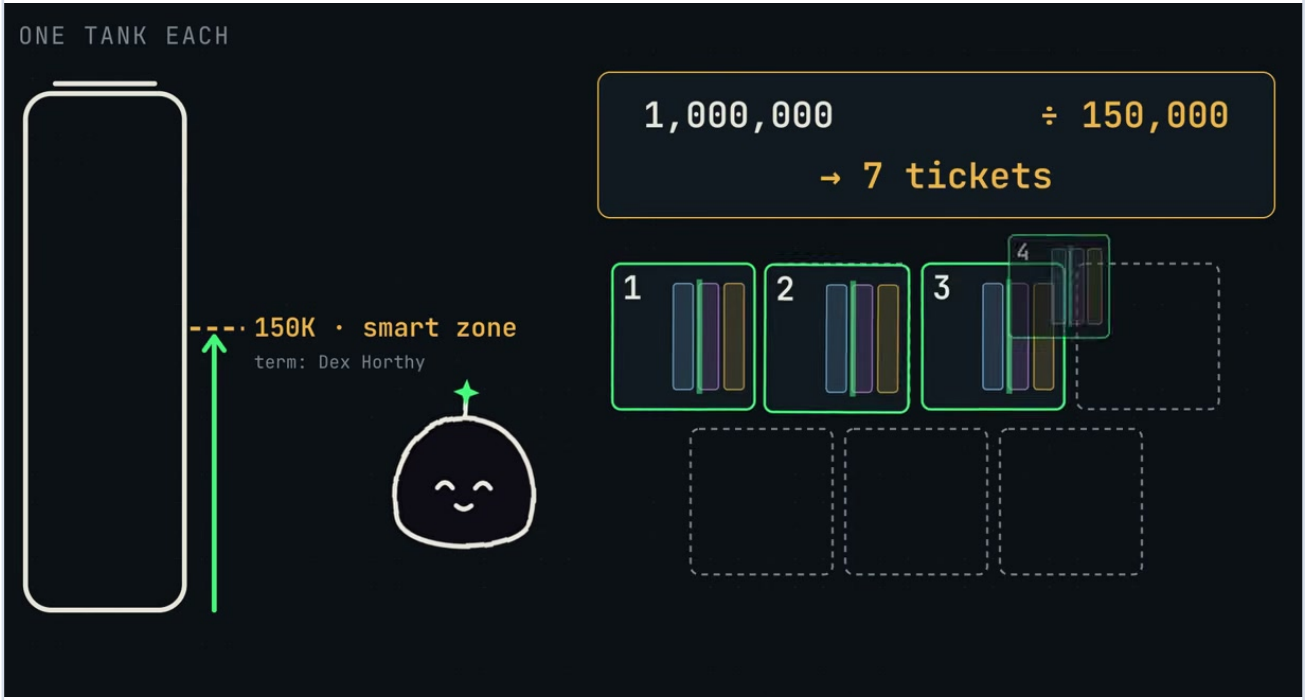
- A ticket is a small, independently implementable slice of the spec with a clear outcome, dependencies, and proof of completion.
- Run `/to-tickets` to divide the destination into vertical slices that each span the needed database, API, and interface layers.
- The example yields charge stopping, resume date, packed-bag behavior, and email tickets; dependencies declare that later slices wait for foundations they need.
- The source calls roughly the first 150K tokens a model's `smart zone`; that threshold is a version-sensitive heuristic, not a universal model limit.

CORE IDEA

Vertical tickets keep each implementation coherent while bounding cognitive load.

Size Work by Context, Not Calendar

Estimate tokens, divide by the working-context heuristic, then verify dependencies



NOTES

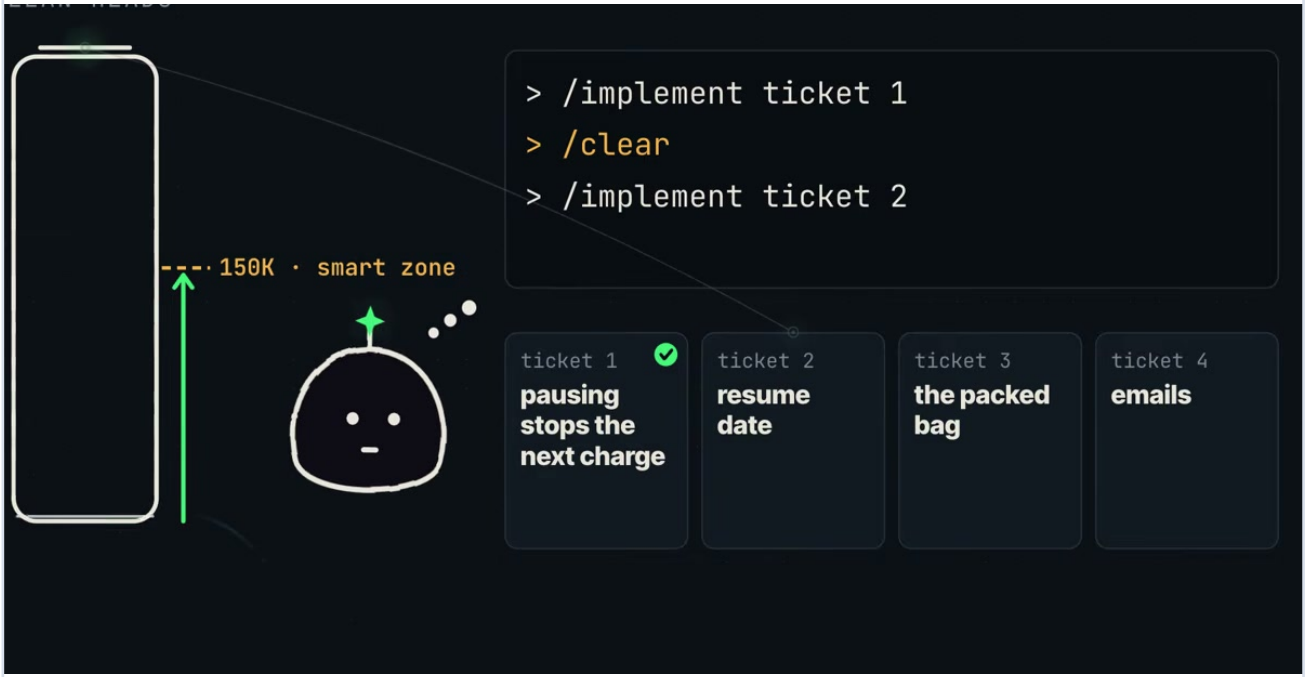
- The shown sizing rule is `estimated tokens ÷ 150,000 ≈ ticket count`; a one-million-token refactor therefore becomes about seven tickets.
- Use the result as a first cut, then adjust for vertical completeness, natural boundaries, testability, and dependency order.
- Do not treat token estimates as precision planning: model, harness, repository, and task type can shift the useful context window.
- A ticket that cannot produce a working slice on its own is probably organized by technical layer rather than user value.

CORE IDEA

Context-aware sizing protects reasoning quality without pretending the heuristic is exact.

Start Every Ticket With a Clean Head

Implement, clear, and reload only the next slice



NOTES

- A fresh context is a new session that carries only the ticket, relevant repository state, and necessary instructions—not the reasoning residue of earlier tickets.
- The manual sequence is `/implement ticket 1`, `/clear`, then `/implement ticket 2`; repeat for each ready ticket.
- Each slice starts with a failing test at the agreed seam, followed by only enough code to make that test pass.
- Clearing context is not deleting repository history: completed code and tests remain, while conversational baggage is discarded.

CORE IDEA

Clean contexts prevent one ticket's assumptions from contaminating the next.

Parallelize Ready Tickets in Isolation

Use /implement-spec to coordinate separate repository copies



NOTES

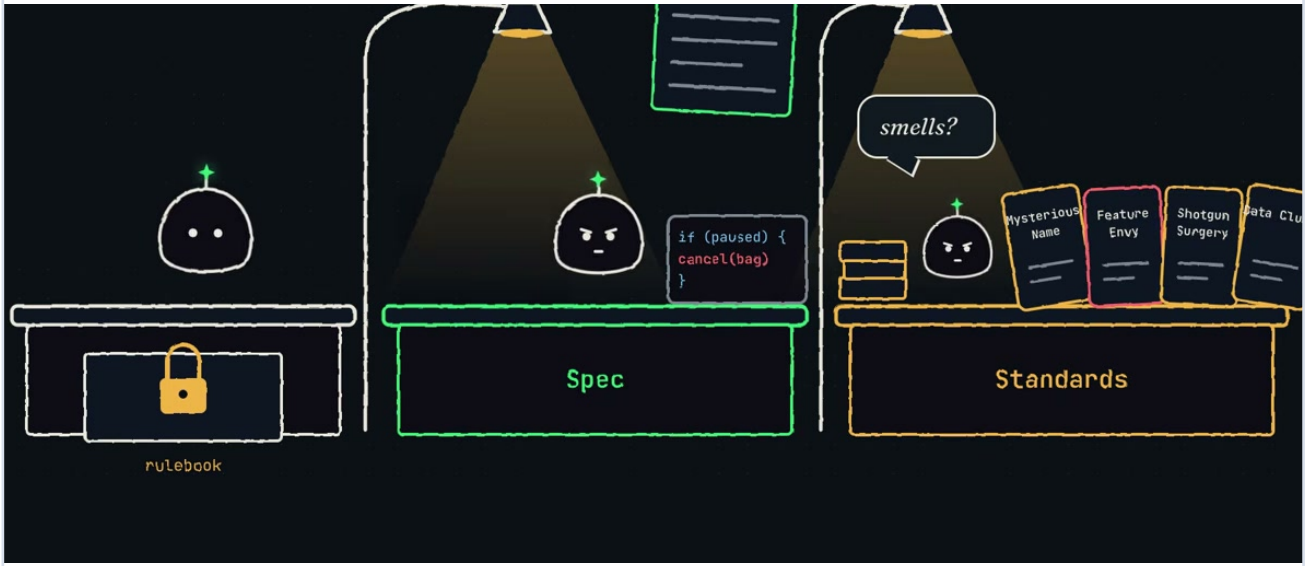
- `/implement-spec` launches one agent for each dependency-ready ticket, each with its own fresh context and isolated copy of the repository.
- Independent tickets can run in parallel; blocked tickets wait until prerequisites land.
- The illustrated workflow collects all results onto one branch, so integration remains a deliberate coordination step rather than an accidental shared workspace.
- Parallelism increases throughput but also raises integration risk; use explicit dependencies, tests, and review rather than assuming isolated successes compose cleanly.

CORE IDEA

Isolation enables parallel work without sharing a muddled context.

Review at Two Desks

Fresh reviewers separately inspect intent and code quality



NOTES

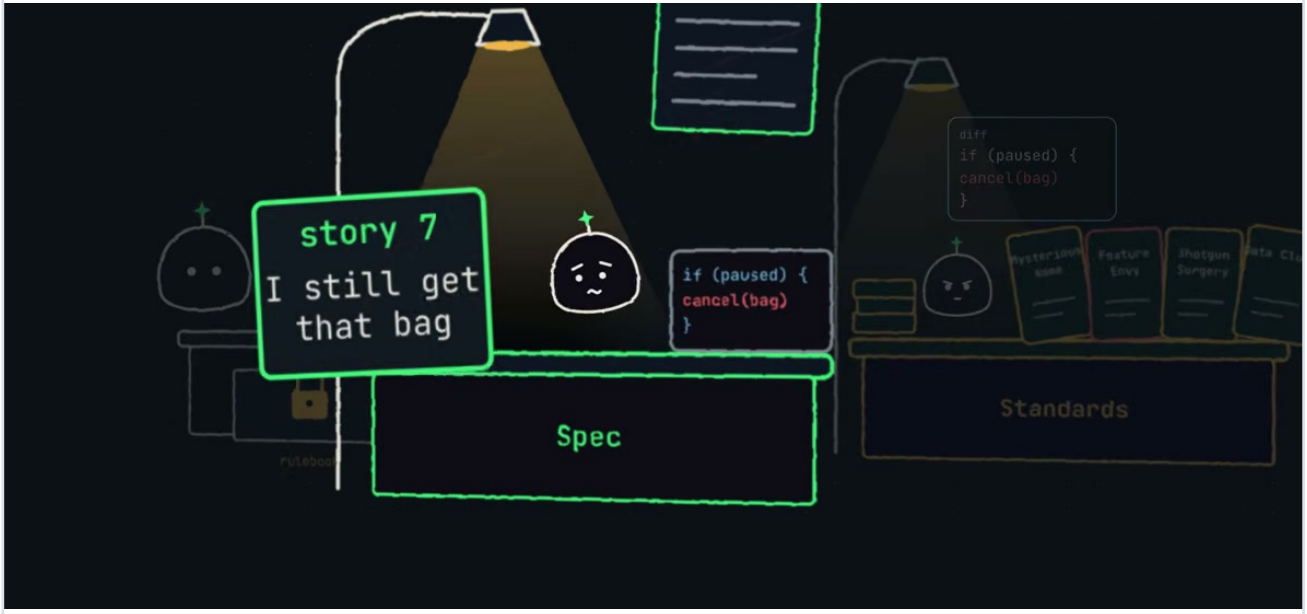
- Two-desk review is a split review in which one fresh agent checks the diff against the spec and another checks it against coding standards and known code smells.
- The spec desk looks for missing behavior, unrequested additions, and changes that appear complete but violate a user story.
- The standards desk focuses on structure and maintainability; refactoring belongs here because the implementation loop was shown to skip a dedicated refactor step.
- The original implementer does not perform either review, reducing self-justification after writing the code.

CORE IDEA

Separating intent review from quality review gives each critic one job.

Quote the Broken Story

Tie each defect to the exact requirement it violates



NOTES

- The spec reviewer cites story 7—`I still get that bag`—instead of merely reporting that the implementation feels wrong.
- The discovered defect cancels the already-packed bag when the customer pauses, directly contradicting the accepted outcome.
- Quoting the story supplies an objective correction target and a regression test, keeping review anchored to prior decisions.
- If no story covers the disputed behavior, return to product discovery rather than allowing review to invent policy.

CORE IDEA

Requirement-linked defects are easier to fix and harder to argue past.

Make Merge Risk Explicit

The PR pairs evidence with reversibility and blast radius



NOTES

- A PR, or pull request, is the reviewable change set proposed for merging into the shared codebase.
- Run `/pr` to produce a concise description with the smallest useful diagram, before-and-after evidence, and a `Merge Danger` section.
- Classify the change as a one-way door when reversal is difficult or a two-way door when it is readily reversible.
- Blast radius means the systems, users, or operations that could be affected if the change fails; adding a subscription column is classified here as a two-way door with billing as the blast radius.
- Risk labels are judgments to verify against migrations, rollback behavior, and deployment order—not guarantees created by the template.

CORE IDEA

A merge decision needs evidence about failure and recovery, not just a code diff.

Run a Retro Before Clearing Context

Mine hidden friction from the completed session



NOTES

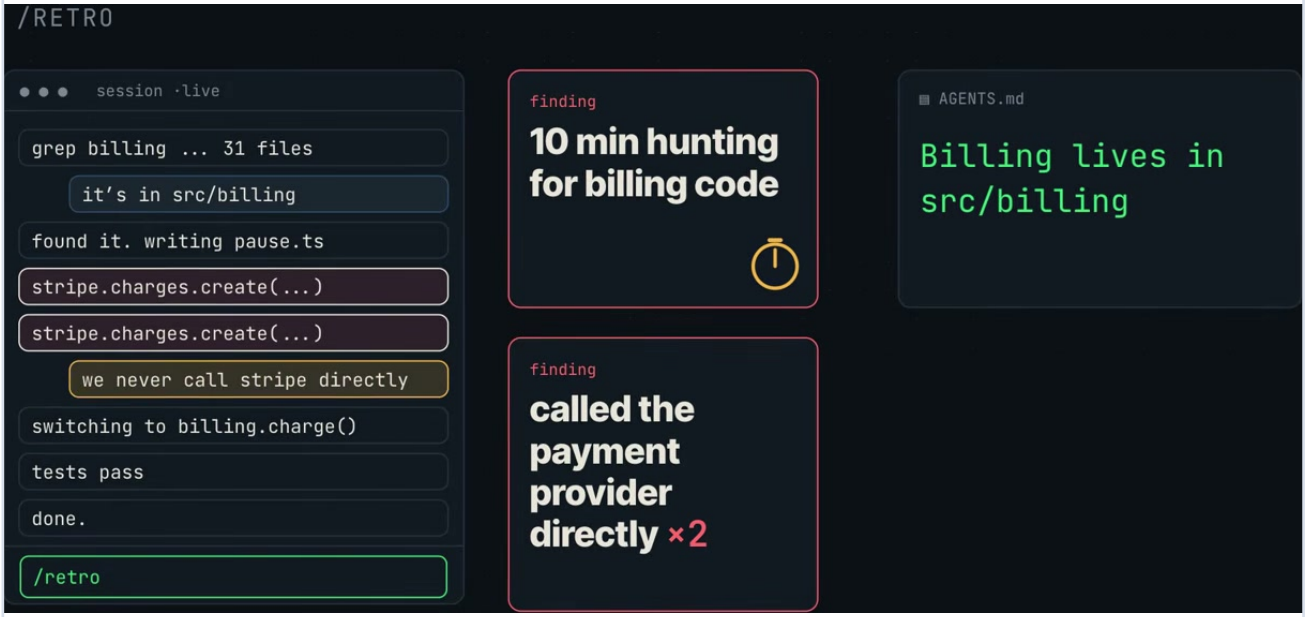
- A retro is a retrospective pass over the just-finished session that proposes improvements to the repository, automated checks, and agent instructions.
- The concrete command is `/retro`, run before `/clear` while search detours, corrections, and failed assumptions are still present in context.
- Agents often push through friction without surfacing it; the retro converts buried effort into candidate system improvements.
- The output is a proposal list, not permission to edit the environment automatically.

CORE IDEA

The session contains operational knowledge that disappears when context is cleared.

Convert Friction Into Repository Memory

Point future agents to the right code and block repeated misuse



NOTES

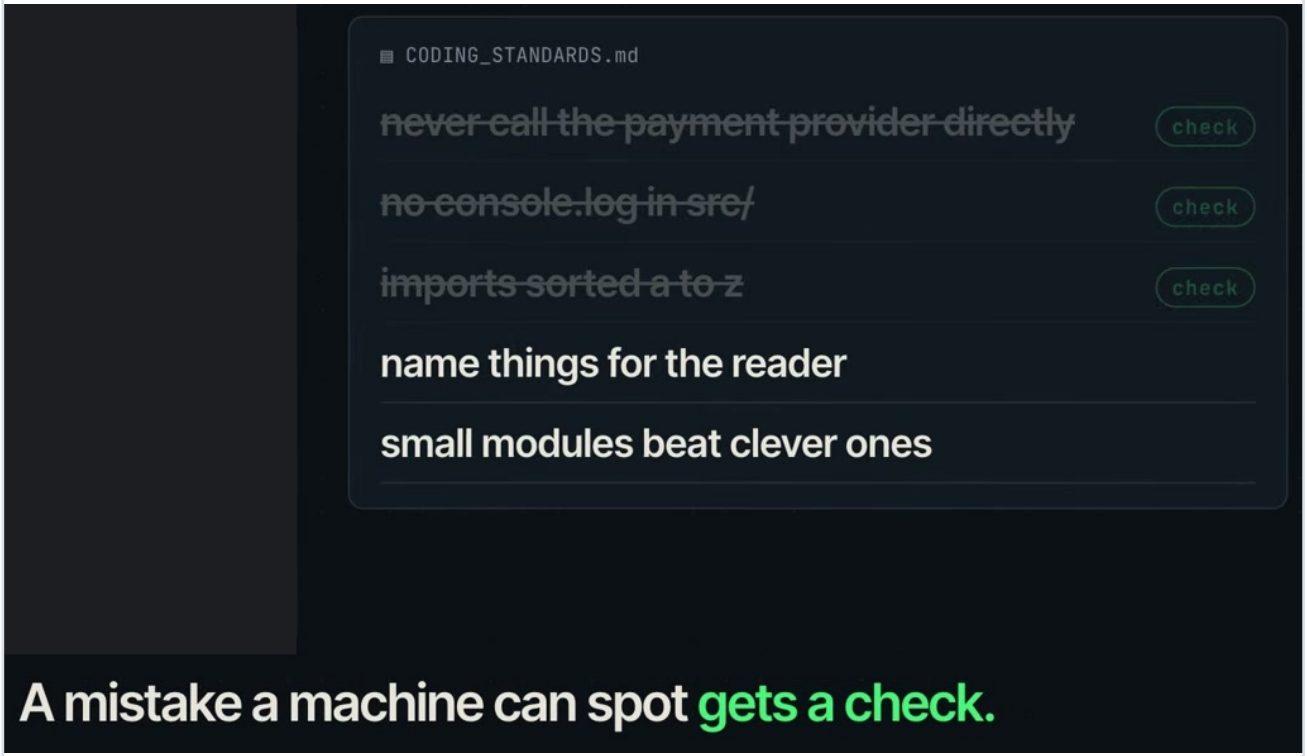
- Ten minutes spent locating billing code becomes a one-line instruction such as `Billing lives in src/billing` in `AGENTS.md`.
- Calling the payment provider directly twice becomes a candidate enforceable rule because the repository expects calls through `billing.charge()`.
- Store navigation facts as instructions and recurring machine-detectable violations as automated checks.
- A retro should improve the environment for the next ticket rather than merely describe what went wrong.

CORE IDEA

Small, durable guardrails compound across future sessions.

Automate What a Machine Can Judge

Reserve written standards for decisions that require judgment



NOTES

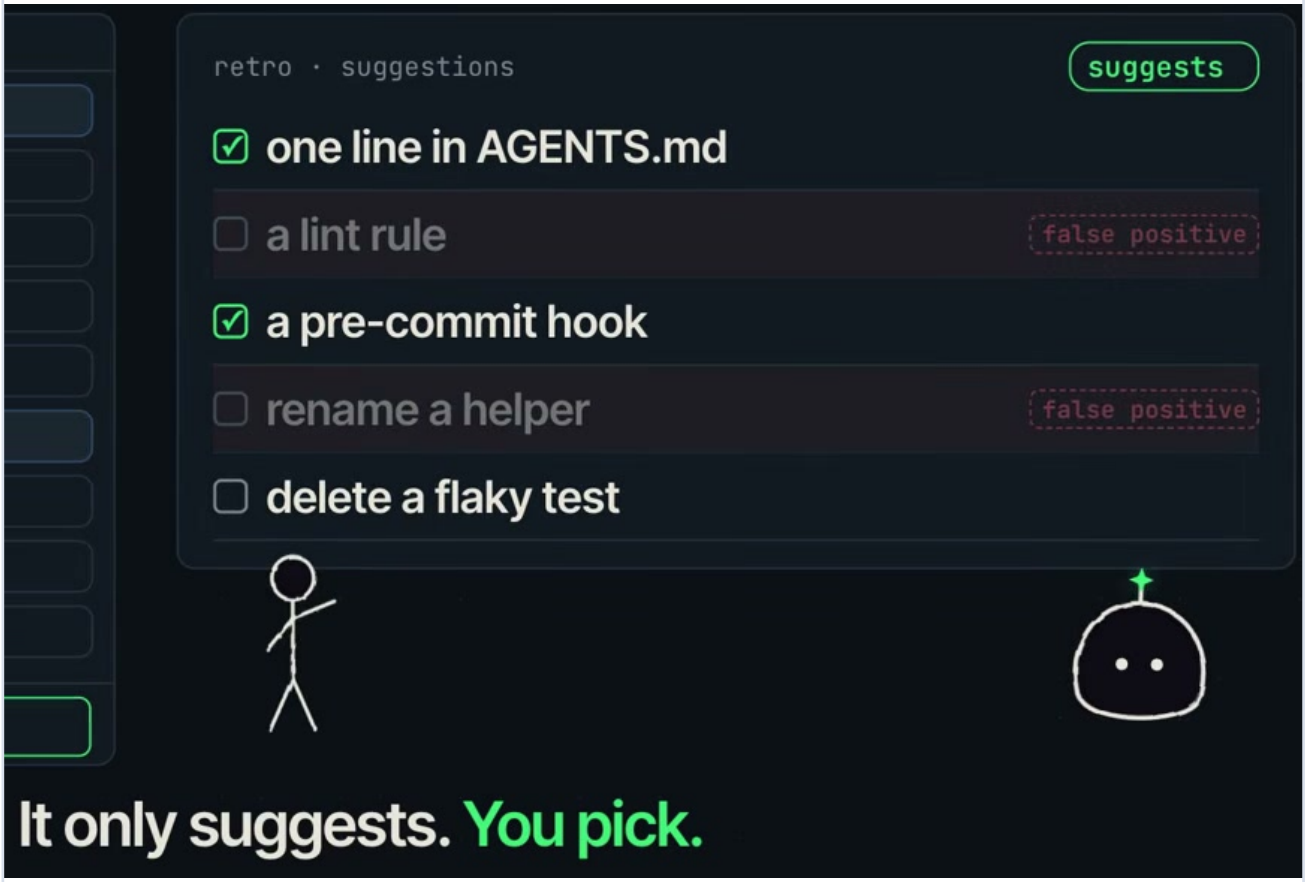
- Turn objective rules—no direct payment-provider calls, no `console.log` in `src/`, sorted imports—into lint rules, hooks, or tests.
- Keep qualitative guidance such as reader-centered naming and preferring small modules in the written standards.
- Decision rule: if a machine can detect the mistake reliably, enforce it mechanically; if context and tradeoffs matter, keep it reviewable guidance.
- Do not automate a noisy detector merely because it is possible; false positives train teams to ignore checks.

CORE IDEA

Deterministic checks prevent known errors without spending review attention repeatedly.

Keep Retro Suggestions Human-Gated

Select useful fixes and reject false positives



NOTES

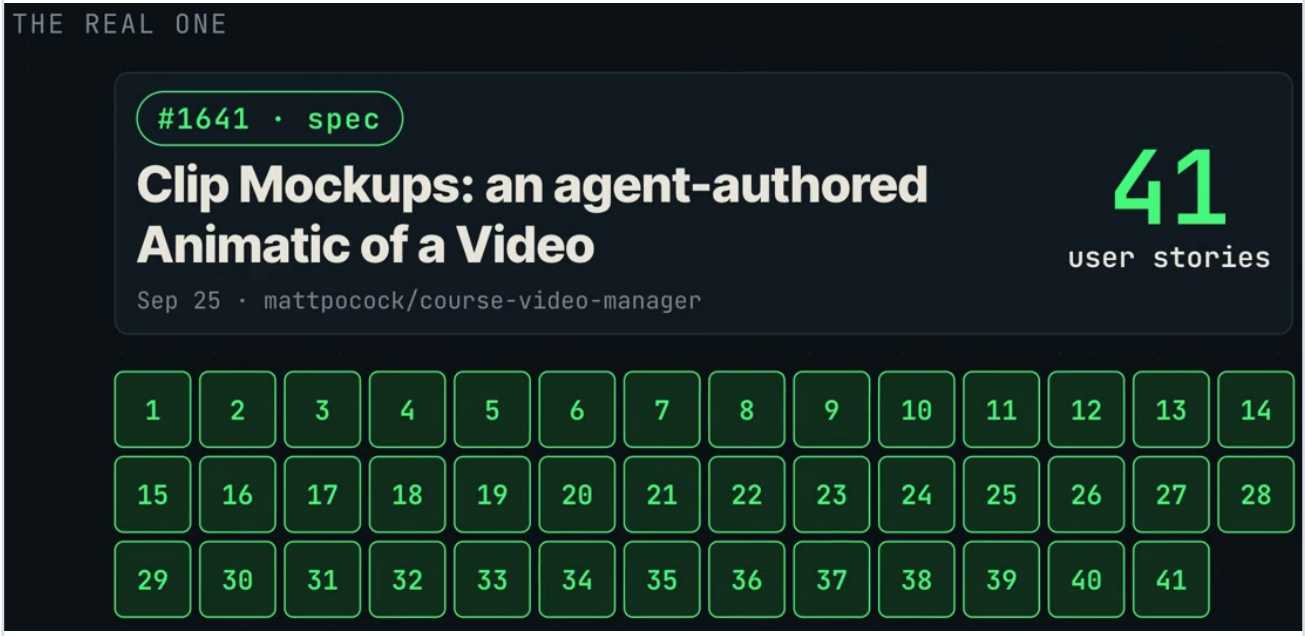
- The retro may suggest an instruction, pre-commit hook, lint rule, rename, or test deletion; each proposal has a different risk profile.
- Accept only changes supported by the session evidence and repository conventions; mark weak proposals as false positives.
- Do not put the loop on autopilot: an agent applying its own findings can chase spurious issues and damage useful code or tests.
- The governing rule is `it suggests; you pick`, preserving human control over lasting safeguards.

CORE IDEA

Self-improvement is safer when diagnosis is automated but policy remains reviewed.

Case Study: 41 Stories

A real repository spec, not the coffee illustration



NOTES

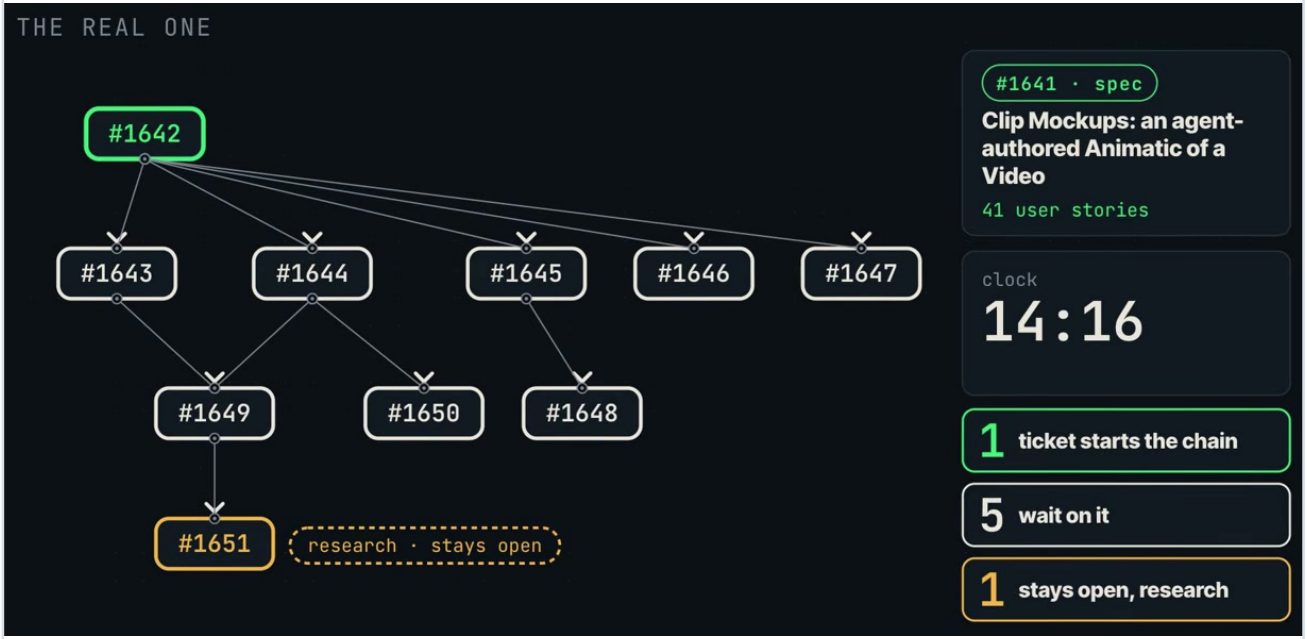
- The source-reported case study is issue `#1641` in `mattpocock/course-video-manager`, dated September 25, for agent-authored animatic clip mockups.
- The spec contains 41 user stories, demonstrating the workflow on a repository task larger than the four-ticket coffee illustration.
- These figures document one observed run; they are not a benchmark for expected story count, quality, speed, or cost.
- The separation matters: the coffee feature explains the process, while this repository history supplies the real execution evidence.

CORE IDEA

A real trace shows the workflow's shape without turning one run into a universal claim.

Ten Tickets Form a Dependency Graph

One starts the chain, five wait, and research stays open



NOTES

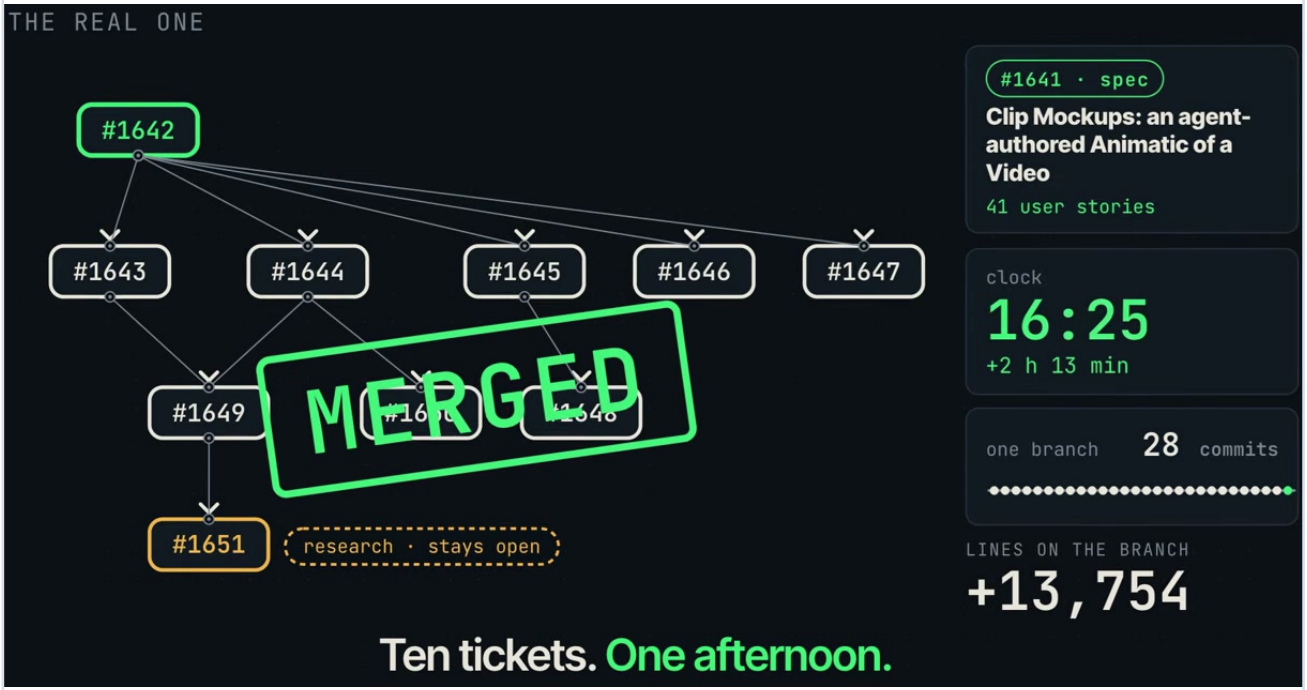
- Within four source-reported minutes, the 41-story spec was decomposed into ten linked tickets.
- Ticket `#1642` starts the chain; five tickets wait on it, downstream tickets unlock later, and `#1651` remains open intentionally as research.
- Explicit blockers tell `/implement-spec` which work is safe to parallelize and which work must wait.
- Research can remain unresolved without pretending it is implementation-ready or blocking unrelated deliverables.

CORE IDEA

Dependencies turn parallel execution from guesswork into controlled scheduling.

One Branch, One Afternoon

Source-reported output from the 10-ticket run



NOTES

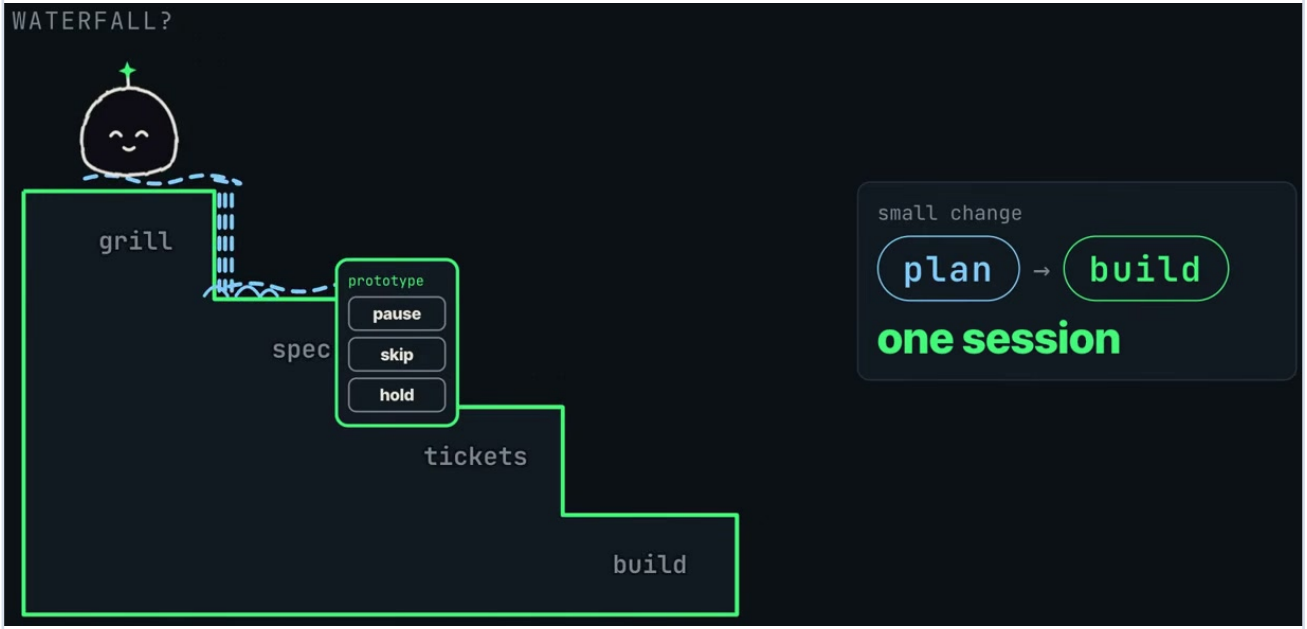
- The reported branch merged 2 hours 13 minutes after the spec timestamp, with ten tickets coordinated onto one branch.
- The branch contained 28 commits and 13,754 added lines; those are repository-history figures, not an independent productivity benchmark.
- A high line count does not establish correctness or maintainability; the supporting controls are dependency handling, tests, fresh review, and explicit merge risk.
- Five days later, a subsequent change included its own Merge Danger section, showing the risk-reporting practice persisted beyond the illustrated run.

CORE IDEA

Throughput claims are meaningful only when paired with traceable process and caveats.

Specification Is Not a Permanent Phase Gate

Prototypes shorten feedback, and small changes stay in one session



NOTES

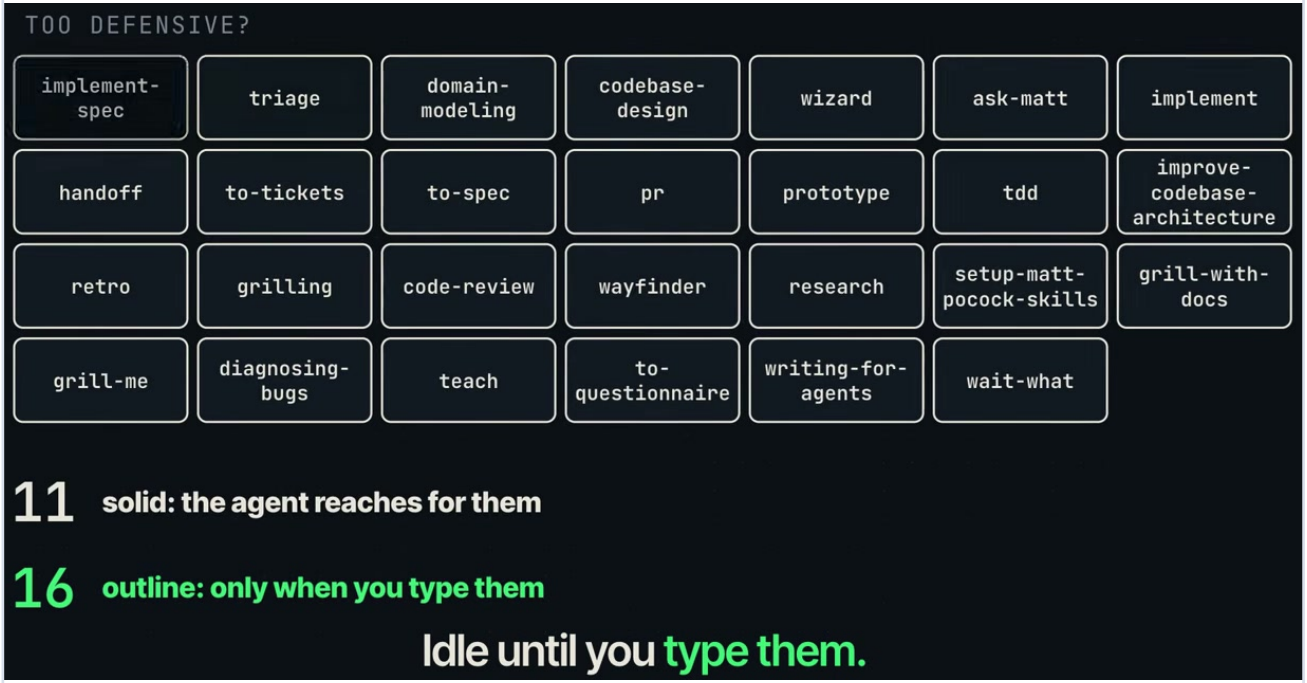
- The waterfall objection sees grill → spec → tickets → build as rigid sequential phases.
- In this workflow, the spec is a projection of discovery decisions; after shipping, the issue closes and durable truth moves into code, tests, and the glossary.
- Throwaway prototypes provide feedback before commitment, reducing the risk of finalizing visual or interaction decisions on paper.
- Selective rule: if the change fits safely in one context, plan and build it in one session rather than manufacturing tickets and handoffs.

CORE IDEA

Use structure to manage uncertainty, not to create ceremony.

Safeguards Can Stay Dormant

Invoke heavier skills only when the task earns them



NOTES

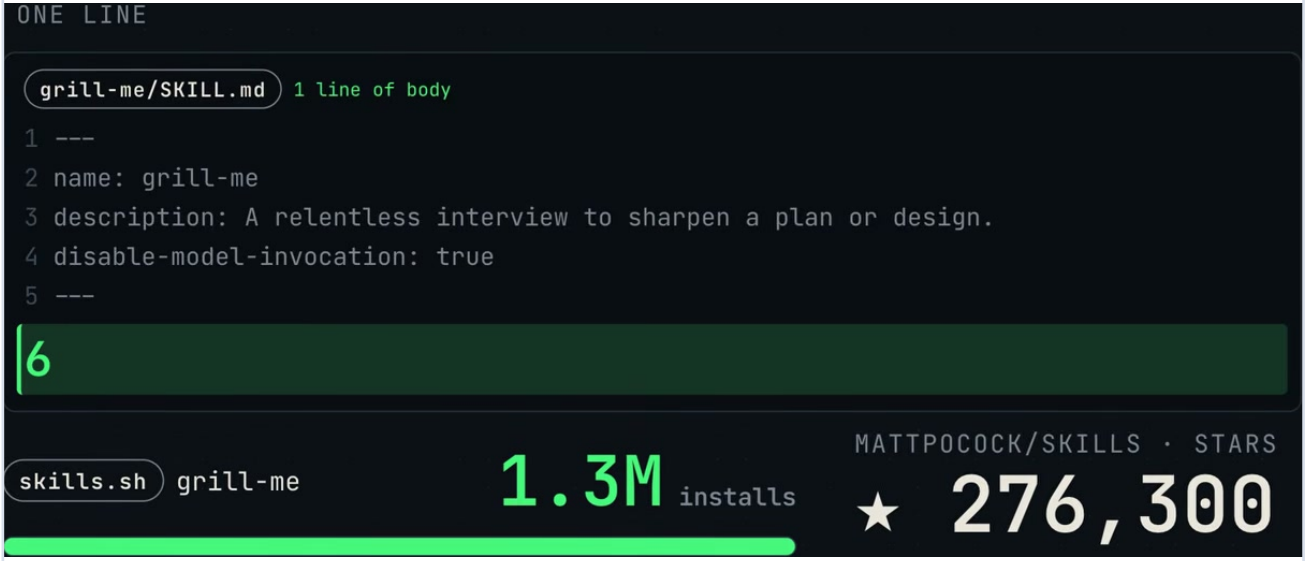
- The objection is that capable models may not need so much scaffolding on every task.
- In the shown plugin snapshot, 16 of 27 skills are explicit-invocation tools: the agent does not load them until their command is typed; 11 are available for agent selection.
- The installed set was reported at about 660 tokens in one tutorial, but counts and token overhead are version- and harness-sensitive.
- Apply safeguards selectively: discovery for ambiguous policy, ticketing for work that exceeds a clean context, and lightweight direct execution for small reversible changes.
- Remove obsolete safeguards when models or harnesses reliably absorb them; one cited release removed a merge-conflict skill for that reason.

CORE IDEA

Good process is demand-loaded rather than imposed uniformly.

The Skill Can Be One Line

A trigger delegates to reusable behavior instead of duplicating it



NOTES

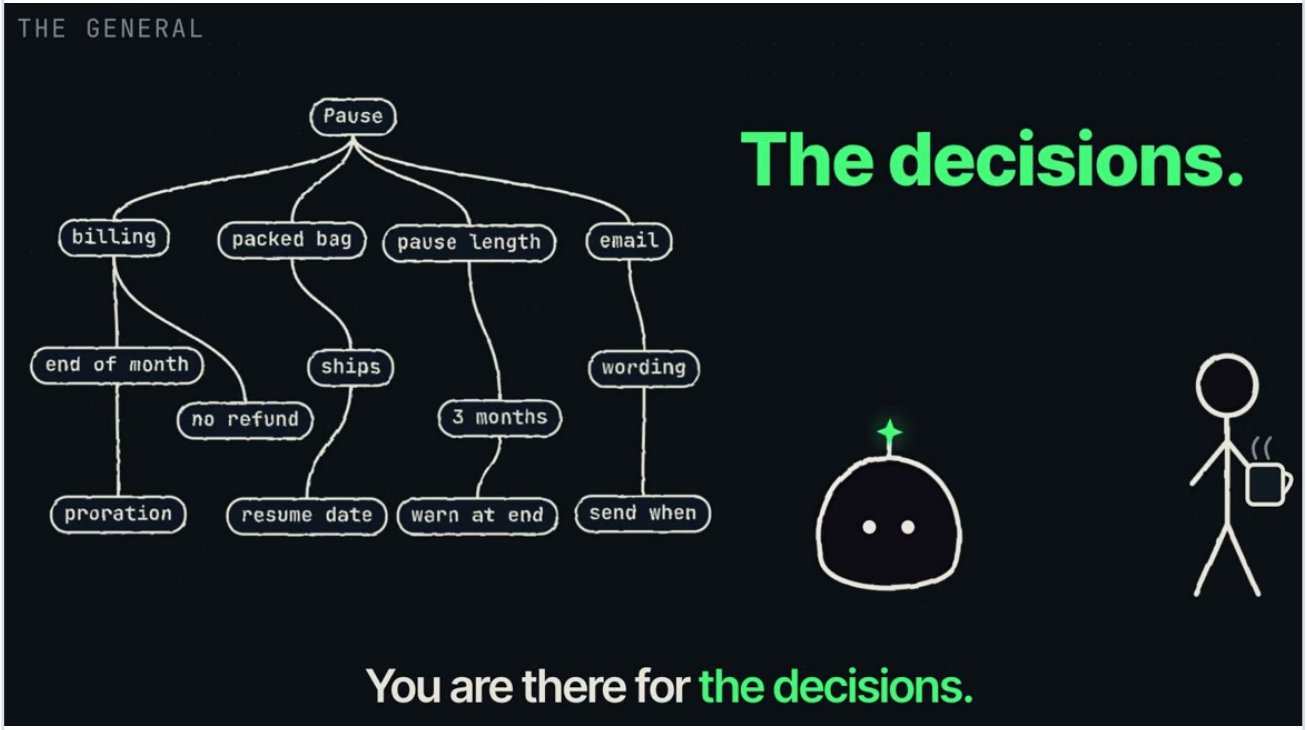
- A skill trigger is a short command definition whose description tells when to invoke a reusable procedure; it need not contain the entire procedure itself.
- The shown `grill-me/SKILL.md` defines the name, describes a relentless interview, disables automatic model invocation, and delegates to the grilling skill.
- Reported install and repository-star figures shown on screen are time-sensitive popularity counters, not evidence of technical quality.
- Keep triggers thin and behavior centralized so improvements propagate without copying long prompts across commands.

CORE IDEA

The leverage lies in the interaction pattern, not prompt bulk.

Own the Strategic Decisions

Agents execute tactics; humans define policy and tradeoffs



NOTES

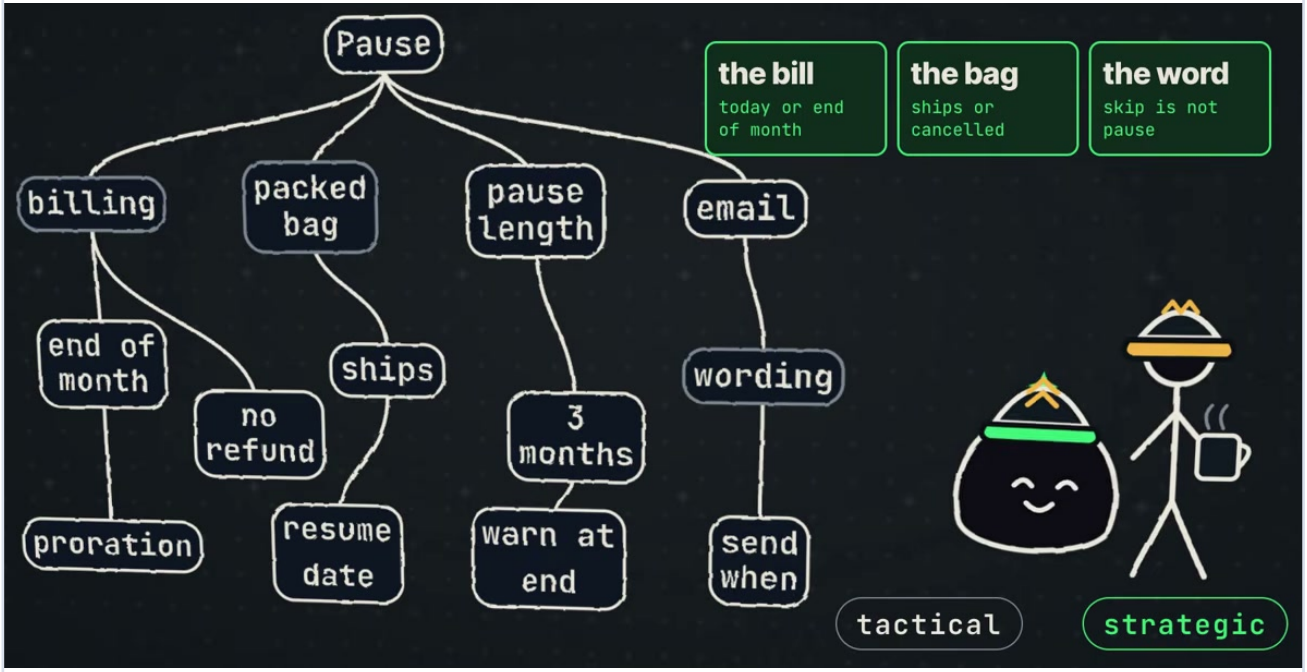
- The enduring human work is choosing billing timing, packed-inventory behavior, pause duration, terminology, reminders, refunds, and other policy branches.
- Repository facts can be delegated to research agents, and implementation can be delegated ticket by ticket, but unresolved value judgments cannot be outsourced safely.
- The tactical/strategic split frames the agent as a strong implementer operating under human intent rather than an autonomous product owner.
- Review, merge classification, and retro acceptance are additional strategic checkpoints where consequences exceed code generation.

CORE IDEA

Delegating code increases the value of clear human judgment.

The End-to-End Control Loop

Decide, specify, slice, isolate, review, merge, and learn



NOTES

- For the illustration, the settled policy is: the already-packed bag ships, billing stops until the resume date, and Skip remains distinct from Pause.
- The complete loop is `/grill-me` → glossary and prototypes as needed → `/to-spec` → `/to-tickets` → fresh-context implementation or `/implement-spec` → two-desk review → `/pr` → `/retro`.
- Use dependencies and isolated repository copies for safe parallelism; use tests at agreed seams and Merge Danger to make correctness and recovery inspectable.
- Select only the safeguards proportionate to ambiguity, context size, reversibility, and blast radius; the workflow is a decision system, not a mandatory command checklist.

CORE IDEA

Reliable autonomy comes from a human-owned decision loop around agent execution.